

Project 3: Probabilistic Optimization

1 Portfolio Layout Optimization

2

2,8.00000000
 3,6.92820323
 4,5.65685425
 5,4.70228202
 6,4.00000000
 7,3.34926665
 8,3.05529777
 9,2.74391668
 10,2.69575702

3

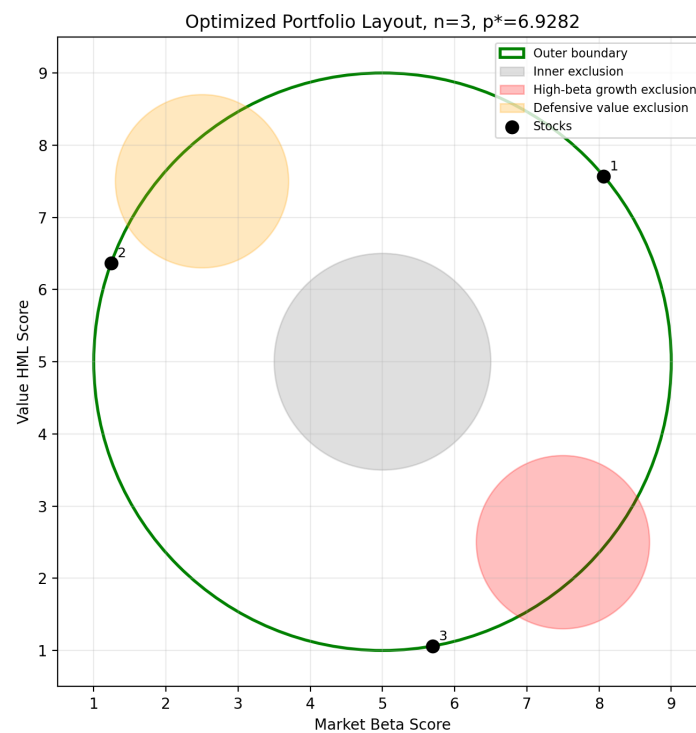
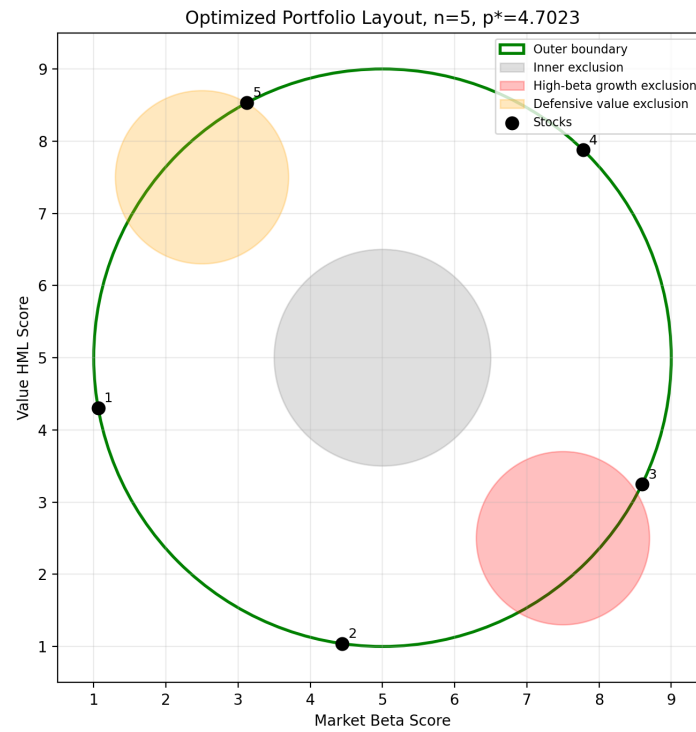
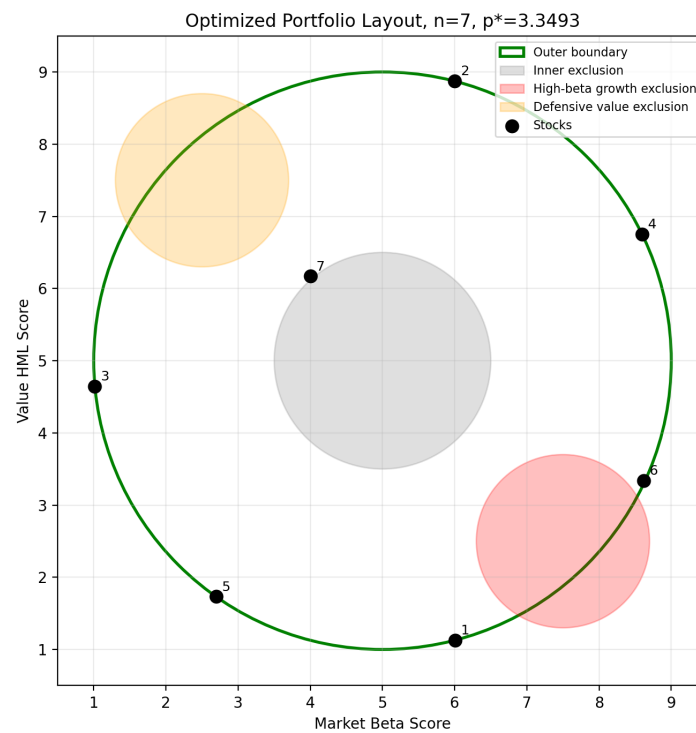
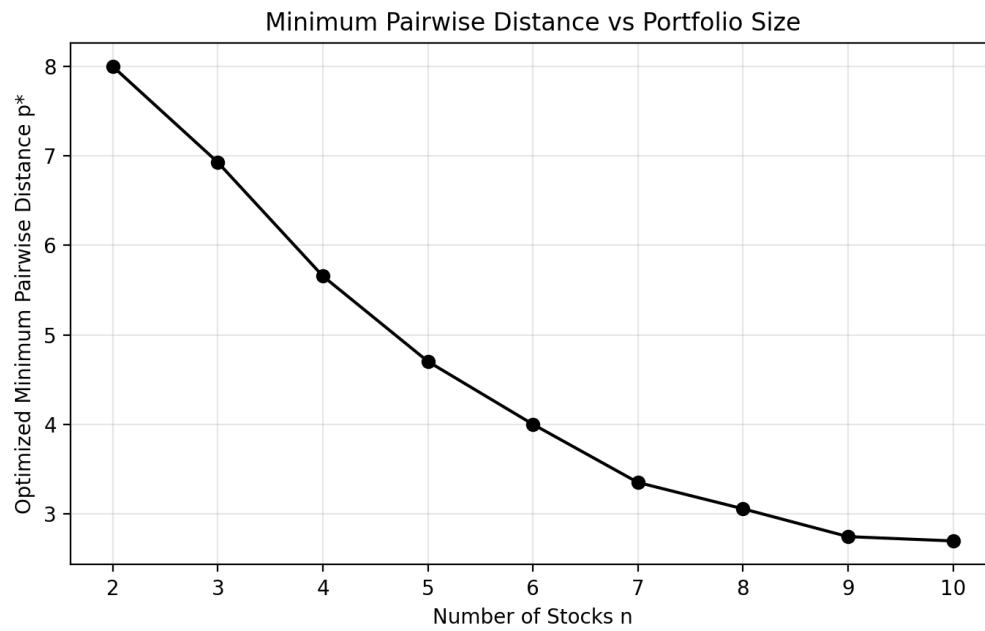


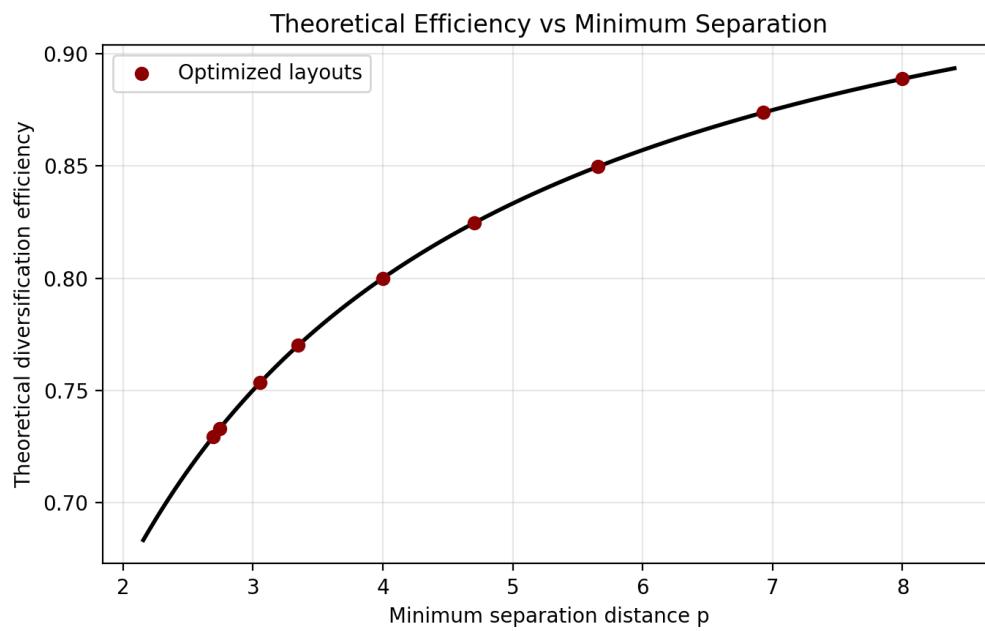
Figure 1: (a) $n = 3$

Figure 2: (b) $n = 5$ Figure 3: (c) $n = 7$

4

Figure 4: Minimum pairwise distance p^* versus number of stocks.

5

Figure 5: Theoretical diversification efficiency as a function of p^*

Code

The implementation is included in Appendix A.1.

2 Gaussian Process Fitting

1

The implementation is included in Appendix A.2

2

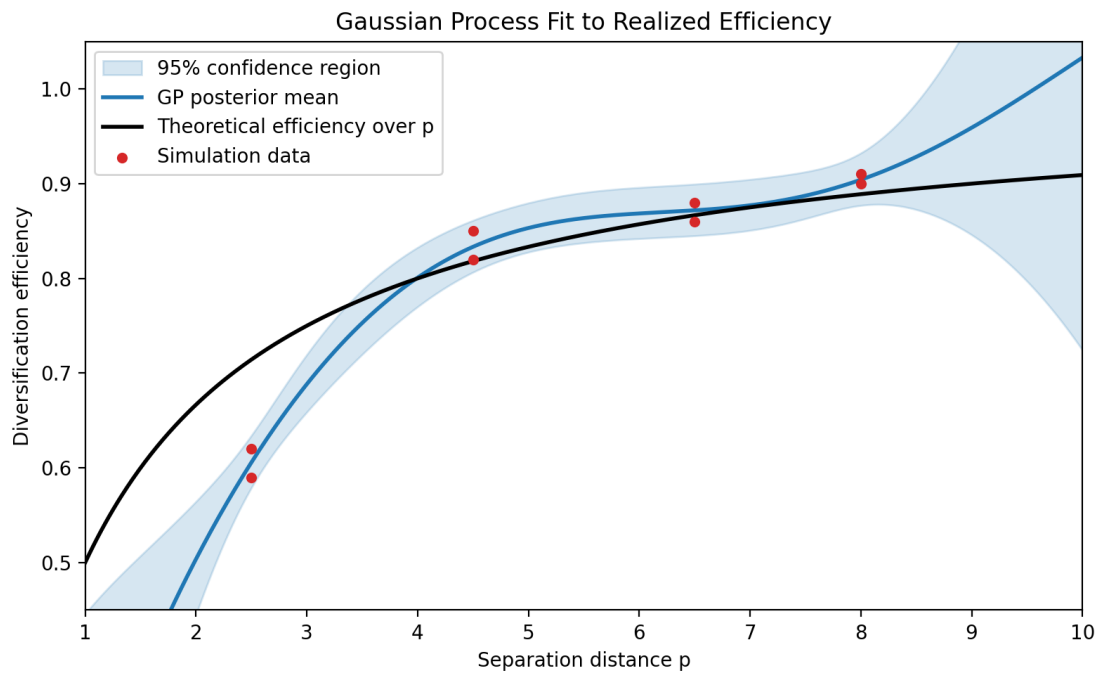


Figure 6: Gaussian process fit for realized diversification efficiency.

3 Optimization Under Uncertainty

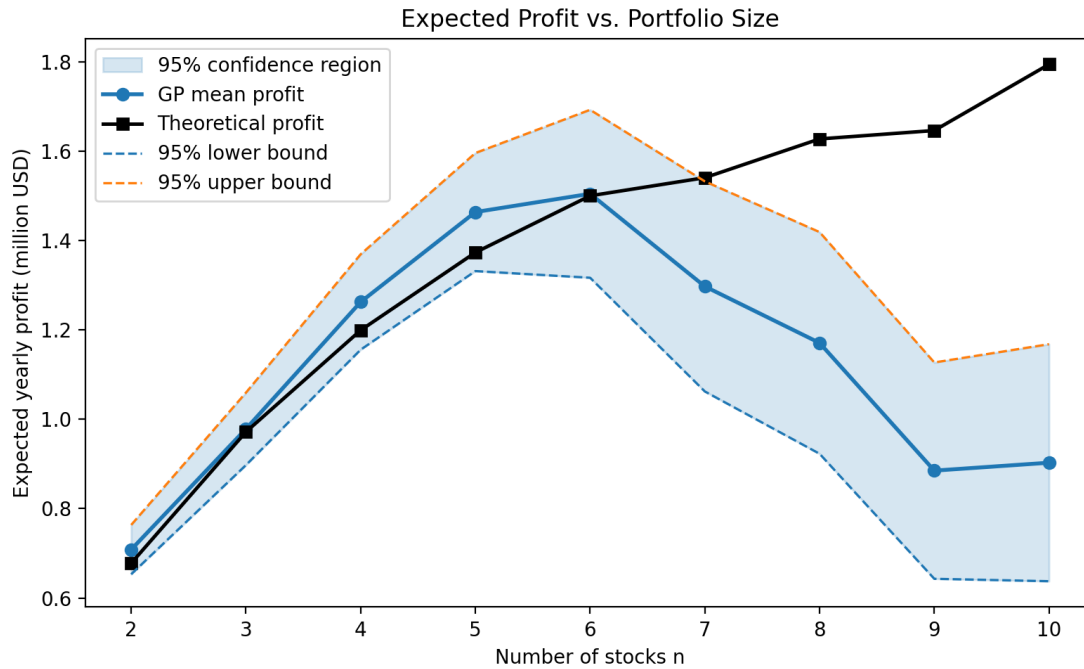


Figure 7: Expected yearly profit as a function of portfolio size.

Recommendation

The recommended portfolio size is $n = 6$. Using only the theoretical diversification curve, the largest expected yearly profit occurs at $n = 10$, because adding more stocks increases the gross return term even though the minimum separation distance decreases. However, the Gaussian-process model learned from the simulation data gives a more conservative efficiency estimate at small separations. Under the GP posterior mean, the best expected profit is at $n = 6$, about \$1.50M per year.

The uncertainty bounds support the same general decision. The lower 95% confidence bound is maximized at $n = 5$, while the upper 95% confidence bound is maximized at $n = 6$. This means the main tradeoff is between $n = 5$ and $n = 6$: $n = 5$ is slightly more conservative, but $n = 6$ has the best posterior-mean and optimistic performance. Therefore, I would choose $n = 6$ unless the manager is strongly risk averse, in which case $n = 5$ is the safer alternative. Additional simulation effort should focus near the corresponding distances $p^* \approx 4.0$ and $p^* \approx 4.7$, where the decision is most sensitive to uncertainty.

Code

The implementation is included in Appendix A.3.

A Code

A.1 Portfolio Layout Optimization

```
1 ZONE_1 = np.array([5.0, 5.0])
2 ZONE_2 = np.array([7.5, 2.5])
3 ZONE_3 = np.array([2.5, 7.5])
4 INNER_RADIUS = 1.5
5 OUTER_RADIUS = 4.0
6 EXCLUSION_RADIUS = 1.2
7
8 def feasible_points(points):
9     d1 = np.linalg.norm(points - ZONE_1, axis=1)
10    d2 = np.linalg.norm(points - ZONE_2, axis=1)
11    d3 = np.linalg.norm(points - ZONE_3, axis=1)
12    return (
13        (d1 >= INNER_RADIUS)
14        & (d1 <= OUTER_RADIUS)
15        & (d2 >= EXCLUSION_RADIUS)
16        & (d3 >= EXCLUSION_RADIUS)
17    )
18
19 def min_pairwise_distance(x, n):
20     points = x.reshape(n, 2)
21     i, j = np.triu_indices(n, k=1)
22     return np.min(np.linalg.norm(points[i] - points[j], axis=1))
23
24 def layout_score(x, n):
25     points = x.reshape(n, 2)
26     if not np.all(feasible_points(points)):
27         return -np.inf
28     return min_pairwise_distance(x, n)
29
30 def project_point(point, n_passes=5):
31     point = point.copy()
32     for _ in range(n_passes):
33         for center, lower, upper in (
34             (ZONE_1, INNER_RADIUS, OUTER_RADIUS),
35             (ZONE_2, EXCLUSION_RADIUS, np.inf),
36             (ZONE_3, EXCLUSION_RADIUS, np.inf),
37         ):
38             direction = point - center
39             distance = np.linalg.norm(direction)
40             if distance < 1e-12:
41                 direction = np.array([1.0, 0.0])
42                 distance = 1.0
43             if distance < lower:
44                 point = center + direction / distance * lower
45             elif distance > upper:
46                 point = center + direction / distance * upper
47     return point
48
49 def project_layout(x, n):
50     points = x.reshape(n, 2).copy()
```

```
51     valid = feasible_points(points)
52     for i in np.where(~valid)[0]:
53         points[i] = project_point(points[i])
54     return points.reshape(2 * n)
55
56 def boundary_candidate_points(rng):
57     candidates = np.vstack([
58         circle_points(ZONE_1, OUTER_RADIUS, 1000),
59         circle_points(ZONE_1, INNER_RADIUS, 1000),
60         annulus_ring_points(n_rings=6, points_per_ring=240),
61         circle_points(ZONE_2, EXCLUSION_RADIUS, 1000),
62         circle_points(ZONE_3, EXCLUSION_RADIUS, 1000),
63         sample_feasible_points(rng, 5000),
64     ])
65     return candidates[feasible_points(candidates)]
66
67 def greedy_maxmin_layout(candidates, n, start_idx):
68     selected = [candidates[start_idx]]
69     available = np.ones(len(candidates), dtype=bool)
70     available[start_idx] = False
71     nearest_dist = np.linalg.norm(candidates - selected[0], axis=1)
72
73     while len(selected) < n:
74         masked_dist = np.where(available, nearest_dist, -np.inf)
75         next_idx = int(np.argmax(masked_dist))
76         selected.append(candidates[next_idx])
77         available[next_idx] = False
78         new_dist = np.linalg.norm(candidates - candidates[next_idx], axis=1)
79         nearest_dist = np.minimum(nearest_dist, new_dist)
80
81     return np.array(selected).reshape(2 * n)
82
83 def greedy_candidate_layouts(rng, n, n_starts, interior_anchor_fraction=0.8):
84     candidates = boundary_candidate_points(rng)
85     interior = sample_feasible_points(rng, max(4 * n_starts, 1000))
86     interior_start = len(candidates)
87     candidates = np.vstack([candidates, interior])
88
89     anchors = [
90         int(np.argmax(candidates[:, 0])),
91         int(np.argmin(candidates[:, 0])),
92         int(np.argmax(candidates[:, 1])),
93         int(np.argmin(candidates[:, 1])),
94     ]
95     n_extra = max(0, n_starts - len(anchors))
96     n_interior = int(np.ceil(interior_anchor_fraction * n_extra))
97     n_global = n_extra - n_interior
98     interior_idx = np.arange(interior_start, len(candidates))
99
100     if n_interior:
101         anchors.extend(rng.choice(interior_idx, size=n_interior, replace=False))
```



```

102     if n_global:
103         anchors.extend(rng.choice(len(candidates), size=n_global, replace=
            False))
104
105     return [
106         greedy_maxmin_layout(candidates, n, int(idx))
107         for idx in anchors[:n_starts]
108     ]
109
110 def cma_es(f, n, k_max, m, m_elite, rng, sigma0=2.0, x0=None):
111     dim = 2 * n
112     weights = np.log(m_elite + 0.5) - np.log(np.arange(1, m_elite + 1))
113     weights = weights / np.sum(weights)
114     mu_eff = 1.0 / np.sum(weights ** 2)
115
116     c_sigma = (mu_eff + 2.0) / (dim + mu_eff + 5.0)
117     d_sigma = 1.0 + 2.0 * max(
118         0.0, np.sqrt((mu_eff - 1.0) / (dim + 1.0)) - 1.0
119     ) + c_sigma
120     c_c = (4.0 + mu_eff / dim) / (dim + 4.0 + 2.0 * mu_eff / dim)
121     c1 = 2.0 / ((dim + 1.3) ** 2 + mu_eff)
122     c_mu = min(
123         1.0 - c1,
124         2.0 * (mu_eff - 2.0 + 1.0 / mu_eff) / ((dim + 2.0) ** 2 + mu_eff),
125     )
126
127     mean = random_feasible_layout(rng, n) if x0 is None else x0.copy()
128     sigma = sigma0
129     C = np.eye(dim)
130     p_c = np.zeros(dim)
131     p_sigma = np.zeros(dim)
132     expected_norm = np.sqrt(dim) * (1.0 - 1.0 / (4.0 * dim) + 1.0 / (21.0
        * dim ** 2))
133     best_x = mean.copy()
134     best_score = f(best_x)
135
136     for generation in range(k_max):
137         eigenvalues, B = np.linalg.eigh(0.5 * (C + C.T))
138         eigenvalues = np.maximum(eigenvalues, 1e-12)
139         C_sqrt = B @ np.diag(np.sqrt(eigenvalues))
140         C_inv_sqrt = B @ np.diag(1.0 / np.sqrt(eigenvalues)) @ B.T
141
142         samples = mean + sigma * (rng.standard_normal((m, dim)) @ C_sqrt.T
            )
143         samples = np.array([project_layout(sample, n) for sample in
            samples])
144         scores = np.array([f(sample) for sample in samples])
145
146         i_best = np.argmax(scores)
147         if scores[i_best] > best_score:
148             best_score = scores[i_best]
149             best_x = samples[i_best].copy()
150
151         elite = samples[np.argsort(scores)[-m_elite:][::-1]]

```

```

152     old_mean = mean.copy()
153     mean = weights @ elite
154     y_w = (mean - old_mean) / sigma
155
156     p_sigma = (
157         (1.0 - c_sigma) * p_sigma
158         + np.sqrt(c_sigma * (2.0 - c_sigma) * mu_eff) * (C_inv_sqrt @
159             y_w)
160     )
161     norm_p_sigma = np.linalg.norm(p_sigma)
162     h_sigma = norm_p_sigma / np.sqrt(
163         1.0 - (1.0 - c_sigma) ** (2.0 * (generation + 1))
164     ) < (1.4 + 2.0 / (dim + 1.0)) * expected_norm
165     p_c = (
166         (1.0 - c_c) * p_c
167         + h_sigma * np.sqrt(c_c * (2.0 - c_c) * mu_eff) * y_w
168     )
169     elite_y = (elite - old_mean) / sigma
170     rank_mu = sum(weights[i] * np.outer(elite_y[i], elite_y[i]) for i
171         in range(m_elite))
172     C = (
173         (1.0 - c1 - c_mu + (1.0 - h_sigma) * c1 * c_c * (2.0 - c_c)) *
174         C
175         + c1 * np.outer(p_c, p_c)
176         + c_mu * rank_mu
177     )
178     sigma *= np.exp((c_sigma / d_sigma) * (norm_p_sigma /
179         expected_norm - 1.0))
180
181     return best_x, best_score
182
183 def slsqp_constraints(z, n):
184     points = z[:-1].reshape(n, 2)
185     p = z[-1]
186     d1 = np.linalg.norm(points - ZONE_1, axis=1)
187     d2 = np.linalg.norm(points - ZONE_2, axis=1)
188     d3 = np.linalg.norm(points - ZONE_3, axis=1)
189
190     constraints = [
191         d1 - INNER_RADIUS,
192         OUTER_RADIUS - d1,
193         d2 - EXCLUSION_RADIUS,
194         d3 - EXCLUSION_RADIUS,
195     ]
196     for i in range(n):
197         for j in range(i + 1, n):
198             constraints.append([np.linalg.norm(points[i] - points[j]) - p
199                 ])
200     return np.concatenate(constraints)
201
202 def refine_layout_slsqp(x0, n, max_steps):
203     start_score = layout_score(x0, n)
204     if start_score == -np.inf:

```

```
201     return x0, -np.inf
202
203     z0 = np.concatenate([x0.copy(), [start_score]])
204     result = minimize(
205         lambda z: -z[-1],
206         z0,
207         method="SLSQP",
208         bounds=[(1.0, 9.0)] * (2 * n) + [(0.0, 8.0)],
209         constraints={"type": "ineq", "fun": lambda z: slsqp_constraints(z,
210             n)},
211         options={"maxiter": max_steps, "ftol": 1e-9, "disp": False},
212     )
213
214     if not result.success:
215         return x0, start_score
216     x = result.x[:-1]
217     score = layout_score(x, n)
218     return (x, score) if score != -np.inf else (x0, start_score)
219
220 def optimize_layout(n, restarts, k_max, m, m_elite, seed, greedy_starts,
221     slsqp_steps):
222     rng = np.random.default_rng(seed)
223     f = lambda x: layout_score(x, n)
224
225     greedy_layouts = greedy_candidate_layouts(rng, n, greedy_starts)
226     ranked_starts = sorted(
227         ((layout_score(x, n), x) for x in greedy_layouts),
228         key=lambda item: item[0],
229         reverse=True,
230     )
231
232     best_x = None
233     best_score = -np.inf
234     cma_layouts = []
235     for i in range(restarts):
236         x0 = ranked_starts[i][1] if i < len(ranked_starts) else None
237         x, score = cma_es(f, n, k_max, m, m_elite, rng, x0=x0)
238         cma_layouts.append(x)
239         if score > best_score:
240             best_x, best_score = x, score
241
242     for x0 in cma_layouts:
243         x, score = refine_layout_slsqp(x0, n, slsqp_steps)
244         if score > best_score:
245             best_x, best_score = x, score
246
247     return best_x, best_score
```

A.2 Gaussian Process

```
1 LENGTH_SCALE = 4.0
2 NOISE_VARIANCE = 0.02 ** 2
3
4 def theoretical_efficiency(p):
5     return 1.0 / (1.0 + 1.0 / p)
6
7 def squared_exponential_kernel(x1, x2, length_scale=LENGTH_SCALE):
8     x1 = np.asarray(x1).reshape(-1, 1)
9     x2 = np.asarray(x2).reshape(1, -1)
10    return np.exp(-0.5 * ((x1 - x2) / length_scale) ** 2)
11
12 def fit_gp_predict(x_train, y_train, x_test):
13     mean_train = theoretical_efficiency(x_train)
14     mean_test = theoretical_efficiency(x_test)
15
16     k_pred = squared_exponential_kernel(x_test, x_test)
17     k_pred_train = squared_exponential_kernel(x_test, x_train)
18     k_train = squared_exponential_kernel(x_train, x_train)
19     noisy_covariance = k_train + NOISE_VARIANCE * np.eye(len(x_train))
20
21     k_prod_inv = np.linalg.solve(noisy_covariance.T, k_pred_train.T).T
22     posterior_mean = mean_test + k_prod_inv @ (y_train - mean_train)
23     posterior_cov = k_pred - k_prod_inv @ k_pred_train.T
24     posterior_variance = np.maximum(np.diag(posterior_cov), np.finfo(float)
25                                     .eps)
26
27     return posterior_mean, np.sqrt(posterior_variance)
```

A.3 Expected Profit

```
1 from gaussian_process import fit_gp_predict, theoretical_efficiency
2
3 CAPITAL_PER_STOCK = 25_000_000.0
4 ALPHA_PER_STOCK = 0.04
5 COST_PER_STOCK = 550_000.0
6 X_TRAIN = np.array([2.5, 2.5, 4.5, 4.5, 6.5, 6.5, 8.0, 8.0])
7 Y_TRAIN = np.array([0.59, 0.62, 0.82, 0.85, 0.86, 0.88, 0.90, 0.91])
8
9 def profit(n, efficiency):
10     return n * efficiency * CAPITAL_PER_STOCK * ALPHA_PER_STOCK - n *
11         COST_PER_STOCK
12
13 def choose_portfolio_size(n_values, distances):
14     gp_mean, gp_std = fit_gp_predict(X_TRAIN, Y_TRAIN, distances)
15     gp_lower = gp_mean - 1.96 * gp_std
16     gp_upper = gp_mean + 1.96 * gp_std
17     theory = theoretical_efficiency(distances)
18
19     theory_profit = profit(n_values, theory)
20     mean_profit = profit(n_values, gp_mean)
21     lower_profit = profit(n_values, gp_lower)
22     upper_profit = profit(n_values, gp_upper)
```